Agentic Edge–Cloud Orchestration for Resource-Efficient Daily Inventory Monitoring with Egocentric Video

Anonymous Author(s)

Abstract

We target automatic, per-item inventory tracking in home kitchens from first-person video. Given a participant's current inventory (items with known starting amounts) and a cloud-uploaded kitchen-activity video, the system outputs, for each inventory item, how much was consumed and how much remains, under an objective of *minimizing VLM cost subject to an accuracy requirement.*

1 Introduction

Wearable AI systems are emerging as a promising platform for augmenting human memory in everyday life. Imagine a user standing in a grocery store and asking their smartglasses, "Do I still have eggs at home?" Answering this question requires more than recognizing objects in the current scene: the assistant must hold a *persistent memory* of prior recordings (the eggs were last seen days ago, not in this aisle), maintain an *up-to-date inventory state* that integrates intervening purchases and uses, and perform *state-change reasoning* that distinguishes acquiring, storing, consuming, and discarding the same item across time. Such capability could address common and consequential challenges in daily food management. In the United States, 30–40% of the food supply is wasted, costing an average family of four approximately 1, 500 per year, while foodborne illness affects an estimated 48 million people annually and causes 128, 000 hospitalizations and 3, 000 deaths []. Better household awareness of *what is on hand and how long it has been there* mitigates both: knowing what is in stock prevents over-purchase and waste, and knowing how recently each item was opened or replenished flags items that should be consumed or discarded before they spoil.

Recently, cloud-based large vision-language models (VLMs) have enabled powerful open-vocabulary visual understanding, making it possible to recognize diverse food items, infer object states, and answer natural-language queries about a scene [18, 19, 23]. A straightforward approach is to continuously stream smartglass video to a cloud VLM and ask it to summarize inventory-related observations. However, enabling practical wearable memory assistance from egocentric video with this design remains highly challenging. *First*, continuous VLM reasoning incurs substantial inference cost. *Second*, inventory memory construction requires long-horizon reasoning to identify meaningful inventory updates, but VLMs are constrained by their context windows, so simply increasing the amount of video provided is not a scalable solution. *Third*, egocentric video is inherently unstable, redundant, and incomplete, with viewpoint instability, hand occlusions, brief object visibility, and fine-grained pour/scoop motions that are hard to read from any single frame; recovering inventory state from such video further requires *object permanence* across sessions and *temporal reasoning* over state-change events, both of which current VLMs handle unevenly. Consequently, even with a high-end, large-context VLM such as Gemini 3 Pro, continuous VLM reasoning still achieves limited inventory-monitoring accuracy, as we later revealed. At Gemini 3 Pro paid-tier rates [6], processing 1 fps egocentric video (≈3,600 frames per hour) costs $3.45 per hour, exceeding $100 per household per month with only one hour of daily logging.

Existing long-video and egocentric-video systems do not directly solve this setting. Long egocentric-video methods build reusable video memories, active-object records, scene representations, or environment-grounded graphs for later retrieval and reasoning [3, 5, 9, 11]. These methods compress video into textual or structured memories and then answer from those memories with an LLM; without revisiting selected visual evidence with a VLM, they cannot perform the fine-grained state-change reasoning needed to estimate package-level food quantity after a pour, scoop, or partial use. Agentic long-video VLM systems can retrieve and inspect selected segments with an LLM/VLM loop [4, 20, 21], providing a closer interface for visual evidence seeking. However, their methods are typically optimized for video QA, summarization, or temporal localization; they do not explicitly optimize for lowering VLM token cost.

To address these challenges, our key insight is that inventory-relevant changes are sparse in egocentric video. Although smartglasses may capture continuous visual streams, only a small subset of moments correspond to meaningful inventory updates, such as object acquisition, storage, removal, consumption, relocation, or state change. This leads to the central research question of this paper: can lightweight edge intelligence identify the candidate inventory-relevant events that require VLM reasoning, thereby reducing token consumption while improving, or at least not degrading, daily inventory-monitoring accuracy?

We propose *proposed system*, an agentic edge–cloud orchestration framework for resource-efficient daily inventory monitoring using egocentric video. Through the applied lens of kitchen food inventory monitoring, *proposed system* integrates lightweight edge observers, agentic event selection, cloud VLM reasoning, and structured memory updates. Specifically, smartglasses record egocentric video stream and upload it to a household edge endpoint, such as a smartphone, home desktop, or local server. The edge endpoint first runs lightweight computer-vision observers over the full session — a hand–object interaction (HOI) detector, an image-embedding-based item identifier, and a scene tagger — producing compact, timestamped logs of inventory-relevant interactions. An LLM-powered agent then acts as a remote planner over these logs: it decides which short video windows warrant deeper reasoning, dispatches the selected segments to a cloud VLM reasoner, and updates a structured inventory ledger.

2 Introduction

The increasing ubiquity of wearable egocentric cameras enables continuous, passive memory augmentation for daily life logging

and contextual assistance. A critical component of these applications is the ability to track object state changes, as physical states reflect the outcomes of daily actions that humans frequently forget (e.g., determining whether a stove was left on, or quantifying how much of an ingredient was dispensed)[cite]. Because these queries require sophisticated causal and physical reasoning, recent works have increasingly relied on large vision–language models (VLMs) to interpret these dynamic transformations [18, 19, 23].

Yet these VLM-based approaches have been validated predominantly in *procedural settings* — recipe execution, assembly, instruction following — where every action advances the task and state changes saturate the timeline [10, 13]. Daily memory augmentation, however, is fundamentally *selective*: the user maintains a finite set of items of interest (a grocery ledger, a list of appliances, a checklist of medications) and asks about their state across long, unstructured recordings of ordinary life. In this regime, interactions with the queried items are sparse within hours of unrelated activity — yet each interaction still demands the same heavy VLM reasoning that procedural settings rely on. This exposes a critical systems bottleneck: feeding hours of always-on video into VLMs incurs prohibitive API token costs [6, 12] and far exceeds the context window of State-of-the-art VLMs [7, 16, 22], even though the task-relevant moments occupy a small fraction of the timeline. At Gemini 3 Pro paid-tier rates [6], end-to-end inference on 1 fps egocentric video costs $3.45/hour — over $100/household/month at one hour of daily logging. How, then, can we achieve accurate, long-horizon state retrieval over a query set without exhausting the VLM token budgets that this reasoning consumes?

We answer this question through the applied lens of automated kitchen food inventory tracking. Food inventory is a near-universal daily memory-augmentation need: households replenish and consume food on a near-daily cycle, and the gap between what is on hand and what the user remembers drives wasted food, missed nutrition goals, and redundant grocery trips [cite]. It also introduces visual perception challenge — food morphs across fluids, solids, granulars, and discrete countables, each calling for a different combination of image retrieval, action recognition, container-aware detection, and volume- or count-based reasoning — exactly the heavy VLM capabilities whose token cost is the bottleneck identified above. Concretely, given untrimmed egocentric video of cooking activities and an initial ledger, our goal is to quantify the consumed and remaining amount of each inventory item while minimizing the VLM token cost subject to accuracy requirement.

We propose *proposed system*, which separates inventory tracking into a *locating* stage and a *reasoning* stage. Smart glasses record kitchen video and upload it to a household edge endpoint — a smartphone, a home desktop, or a local server. The endpoint first runs a lightweight CV pipeline locally over the full session, producing a compact, timestamped record of inventory interactions. It then runs an agent that calls a remote LLM planner over this record to decide which short segment windows are worth observing, dispatches those segments to a remote VLM observer, and uses the observer's per-item answers to update the inventory ledger.

Challenges.

- **C1 – Locating inventory-interaction windows in long video.** Kitchen sessions span tens of minutes to hours; only a few-percent slice involves actual contact with a ledger item. The system must surface those moments cheaply, before any heavy VLM reasoning is invoked.
- **C2 – Selecting the few decision-relevant frames within those windows.** Even after C1, most located frames do not answer "how much of item X remains?" — pre-interaction states, hand occlusions, wrong-item grabs, mid-pour blurs all coexist with the one or two frames that actually expose the post-interaction fill level. The system must read as few frames as possible while still landing on the answer-bearing ones.

Our design.

- Inspired by recent work on long egocentric video understanding [cite] and agentic video analytics [cite], we decompose the task into a *lightweight CV pipeline* that builds a compact, item-indexed context of inventory-interaction events, followed by an *LLM-driven agentic loop* that plans which short windows the VLM should observe, reads the VLM's per-item answers, and replans until each item's remaining amount is resolved or a token budget is exhausted.
- C1 is addressed by the CV pipeline: hand–object interaction detection, scene tagging, and item identification jointly produce a per-item temporal event list that locates candidate windows without invoking a VLM on the full video.
- C2 is addressed by the agentic plan–observe loop: a text-only LLM allocates a shared frame budget across items based on the CV evidence; a VLM observes only the planner-selected frames and returns per-item answers; the planner then replans on items whose state remained uncertain.

Dataset.

- Existing egocentric kitchen datasets annotate actions, not persistent item state, so we collect and annotate our own benchmark: in-the-wild and controlled-consumption recordings across multiple households on Meta Ray-Ban Smart Glasses, with per-session ground-truth starting and remaining amounts for every item the participant interacts with.

Contributions (placeholder list).

- A formulation of selective-query inventory tracking from untrimmed egocentric video as a cost–accuracy problem, with VLM token cost as the cost axis.
- *proposed system*: a CV-located, agentically-planned VLM observation pipeline that addresses C1 and C2.
- A new benchmark of in-the-wild and controlled household kitchen video with per-item starting and remaining amounts.
- Cost–accuracy frontier evaluation against a whole-video VLM baseline on the new benchmark, with cross-cuts by item shape category and per-item lifecycle.

3 Background and Motivation

Daily inventory monitoring is an instance of episodic memory retrieval over egocentric video [8], but it has a more constrained notion of state than standard video QA: the system must answer a fixed set of item-state questions over a persistent inventory ledger.

In our setting, the video source is a natural, long-horizon cooking session, and the state of interest is the food quantity associated with each package instance. We first define the task, describe benchmark construction, and specify the evaluation axes, then situate the problem against object state-change understanding, egocentric video understanding, long-form video retrieval systems, and egocentric kitchen benchmarks.

3.1 Task Definition and Benchmark Construction

Let $V = \{f_t\}_{t=1}^{T}$ denote a natural, untrimmed egocentric recording of a cooking session, and let $L_0 = \{(i, u_i, a_i^0)\}_{i=1}^{N}$ denote the household inventory ledger before the session, where i is an item instance, u_i is its measurement unit (e.g., grams or count), and a_i^0 is its starting amount. The inventory-tracking task is to estimate, for each ledger item, whether the item was used during the session $z_i \in \{0, 1\}$ and, if so, the amount consumed Δ_i and the remaining amount $a_i^T = a_i^0 - \Delta_i$ after the session. If $z_i = 0$, then $\Delta_i = 0$ and the amount is unchanged. Across sessions, the ending ledger becomes the starting ledger for the next recording, so the same item instance is tracked from purchase to exhaustion.

This formulation is fixed-query, long-horizon egocentric VQA [8, 14]: before reading the video, the system knows the set of questions it must answer, namely whether each item $i \in L_0$ was used and how much of it remains. We instantiate the task with a benchmark of natural household cooking sessions. Participants wear egocentric cameras during ordinary meal preparation, register purchased items in a ledger with package size and reference images, and weigh each touched item at the start and end of a session. These measurements produce the ground-truth starting amount, ending amount, and consumed amount for every touched item.

The current benchmark contains recordings from 5 participants, annotated ledgers from 3 participants, 29.9 h of video, 460 weighed consumption events, and 150 tracked item instances spanning fluids, granular foods, solids, and discrete-count items. We evaluate a system by how accurately it updates the ledger and how much cloud VLM token cost it incurs to do so; Section 7 gives the scoring metrics and experiment matrix. Section 6 provides the full collection protocol and item/lifecycle breakdowns after the system design.

3.2 Related Work

Object state-change understanding. A growing literature studies whether objects have been opened, cut, poured, assembled, consumed, or otherwise transformed by human action [17–19, 23]. These works motivate our use of a strong VLM for amount estimation: determining how much sauce was poured or how many tomatoes were sliced is a fine-grained visual reasoning problem. Existing state-change models are typically evaluated on trimmed clips where the relevant interaction has already been isolated. Inventory tracking therefore keeps the reasoning benefit of VLMs, but changes the surrounding problem: the state is a persistent ledger entry for each item, and the answer-bearing moments must first be localized in an untrimmed session before quantity can be estimated.

Egocentric video understanding. Long egocentric videos have structure that general video collections often lack: hands reveal active objects, repeated places anchor daily routines, and the same object can reappear across minutes or sessions. Prior systems exploit this structure by modeling environment affordances in egocentric video, active-object memories in long egocentric streams, or persistent embodied scene memories from egocentric observations and sensors [3, 5, 9, 11]. These works motivate our edge-side per-item interaction log: an HOI detector proposes contact windows, object embeddings link detections to ledger items and package images, and scene embeddings provide storage and use context. Our memory, however, is not a general activity or scene representation. It is keyed by a household inventory ledger and preserves compact evidence about the state of particular package instances.

General long-video retrieval and agents. General long-video systems address context limits by building compact memories, retrieving short evidence spans, or using LLM/VLM agents to choose what to inspect next [4, 9, 15, 20, 21, 24]. VideoAgent-style systems store temporal and object-centric memories for query-specific tool use, while agentic video analytics and active video perception systems iteratively plan observations, read partial evidence, and revise the next frame or segment to inspect. We adopt this agentic locate-then-read pattern after the edge pipeline has produced candidate inventory windows: the planner decides which windows should be sent to the VLM, reads quantity evidence, and revisits unresolved items when the first read is insufficient. The task interface is also different from prior long-video QA: the query set is fixed before the video is read, one question family is repeated for every ledger item, and the output is a persistent per-item state rather than a summary, free-form answer, or temporal interval.

Egocentric video datasets. Large egocentric datasets provide the substrate for action recognition, anticipation, narration grounding, hand-object interaction, segmentation, episodic-memory retrieval, and VQA [1, 2, 8, 14]. EPIC-Kitchens annotates unscripted kitchen videos with verb–noun action segments and retrieval tasks; VISOR adds hand and active-object masks with object relations; Ego4D broadens first-person video to many daily settings with narrations and episodic-memory queries; and HD-EPIC adds dense kitchen annotations such as recipe steps, fine-grained actions, ingredients, object movements, audio events, gaze, and 3D grounding. Procedural cooking datasets such as CaptainCook4D focus on step localization, error recognition, and procedure learning [13]. These benchmarks annotate actions, queries, and object relations, but they do not provide package-level quantities such as the starting amount of a jar, the count of remaining eggs, or the amount used in a session. Nor do they thread the same item instance across multiple sessions to support lifecycle evaluation. Because these labels are exactly what inventory tracking must predict, we collect a new benchmark, described in Section 6, that pairs untrimmed household cooking videos with per-item starting and ending amounts.

4 Motivation Study

We use pilot recordings and existing egocentric kitchen annotations to identify where VLM effort is wasted and motivate *proposed*

system's split between cheap event location and expensive visual reasoning.

VLM cost scales with video length. As a whole-video baseline, we send every sampled frame in a session to Gemini 3.1 Pro and ask the model to infer the final inventory state end-to-end. On a 10-session pilot from our dataset (≈2.2 hours of 1 fps egocentric cooking video, 7,977 frames sent), this baseline costs $7.65 at current paid-tier rates [6] — $3.45 per hour of recording, or over $100/household/month at one hour of daily logging. The cost driver is the linear scaling of input tokens with video length [6, 12]. Any deployable inventory system must therefore avoid sending the full video to the VLM whenever the relevant evidence occupies only a small fraction of the session.

Inventory interactions are sparse. Whether an item was used, and how much was used, can only be resolved around hand–object interactions (HOI) with that item. In a mining pass over HD-EPIC annotations, interactions with inventory-like items account for only roughly 6% of total cooking-video duration; the rest of the timeline includes meal preparation around non-query items, cooking, waiting, washing dishes, and other activity not directly changing the ledger [14]. This sparsity is what makes a locate-then-read split viable: lightweight CV signals can pre-select the seconds in which a ledger item is actually being touched, so an expensive VLM is invoked only on candidate inventory windows.

Irrelevant context degrades amount estimation. Selective reading is necessary for accuracy, not only for cost. When a VLM receives a long egocentric session, the answer-bearing views are embedded among thousands of visually redundant or misleading frames: idle cooking, off-target object handling, hand occlusions, motion blur, and food derivatives that resemble the queried package but no longer reveal its remaining amount. In our pilot whole-video baseline, only $X\%$ of sampled frames contain package-level quantity evidence, and Gemini 3.1 Pro reaches Y CNPE / $Z\%$ event recall when all frames are placed in context, compared with Y' CNPE / $Z'\%$ recall when the prompt is restricted to oracle answer-bearing windows. Thus filling the VLM context with irrelevant noisy frames can both increase token cost and dilute the visual evidence needed for inventory-state updates.

5 System Design

proposed system takes an untrimmed egocentric kitchen video and a per-participant *ledger* (the list of purchased food items with package amounts and reference images), and produces a per-session update of *how much of each item was consumed*. The pipeline is organized as a sequence of numbered stages (scripts in `system_design/`) with well-defined inputs and outputs so that components can be swapped or ablated.

System roles. proposed system comprises four roles, split across edge and cloud:

- **Edge detector pipeline** (stages 1–3 below): runs on the household endpoint — a smartphone, home desktop, or local server paired with the smart glasses. It processes each session video locally and emits a compact, per-item record of inventory interactions.
- **LLM planner** (stage 4a): a text-only language model invoked in the cloud. It reads the per-item record plus the current ledger snapshot and decides which frames the VLM should observe.
- **VLM reasoner** (stage 4b): a strong vision–language model invoked in the cloud, only on the planner-selected frames. It returns per-item answers about confirmation, evidence, and remaining amount.
- **Inventory ledger**: the persistent state. It lives on the edge endpoint, is updated at session boundaries from the reasoner's outputs, and is sent to the cloud only as an in-prompt snapshot for the current session. The cloud LLM and VLM remain stateless across sessions.

5.1 Inputs and assumptions

We assume that each item is registered *at the time of purchase* with two lightweight artifacts:

- A **receipt entry**, supplying the item's identity, quantity, and unit (grams or count). This is what initialises the per-participant ledger; subsequent sessions update an item's remaining amount until it is exhausted.
- One or more **reference images** of the item's package, captured by the participant or pulled from the retailer / web. These are what the item-identification stage (DINOv3, below) matches the in-contact object crops against.

Both artifacts are produced once, when the item enters the household, and reused across every session that touches that item. Within a session, the system assumes nothing else: the video is processed independently against the current ledger snapshot.

5.2 Pipeline stages

(1) **Hand–object interaction (HOI grasp detection).** The Hands23 detector runs at 1 fps across the full video and flags frames in which a hand is in *object-contact* with something it is holding. Only these grasp frames are carried forward; frames with no active grasp are dropped, so downstream stages operate on a sparse set of moments when the participant is actually manipulating an item.

(2) **Scene tagging (OWLv2).** An OWLv2 open-vocabulary detector labels each grasp frame with a coarse kitchen location: `storage`, `sink`, or `stove`. These tags give the planner the lifecycle cue needed to distinguish retrieval from use from put-back when allocating observation budget.

(3) **Item identification (DINOv3).** The in-contact object crop from each grasp frame is matched against the participant's current inventory via DINOv3 image-to-image similarity against each item's reference images (Section 5.1). The matching score decides *which ledger item* is being handled in each grasp frame. A SigLIP2 zero-shot text-to-image score is computed in parallel but is disabled in the canonical configuration, so the planner's evidence rendering relies on HOI contact and DINOv3 alone.

Edge-pipeline output: per-item interaction record. The output of stages 1–3 — the only artifact crossing the edge→cloud boundary,

alongside the ledger snapshot — is a per-item, HOI-gated temporal record. The per-frame grasp, identification, and scene signals are collapsed via heuristic morphological compression (close-then-open on per-frame DINOv3 scores) into a chronological list of *on-contact bursts* per ledger item. Each burst carries: (i) the item it was matched to, (ii) its `[start, end]` timestamps, (iii) the peak DINOv3 match score (the identification confidence used by the planner to weigh evidence), and (iv) the burst's scene-tag distribution (`storage`, `sink`, `stove`) from stage 2. This schema directly addresses challenge C1: it locates per-item candidate evidence windows compactly enough to fit in a single text-only LLM prompt, without ever shipping pixels off the edge.

(4) **Amount estimation.** *proposed system* predicts *how much remained* (or how much was used) for each touched item at the end of the session. This stage is decomposed so that the expensive vision–language reasoning is invoked at most twice per session, each time on a planner-selected frame set:

4a. **Frame-budget planner (text-only LLM).** A text-only LLM reads the session inventory together with the per-item interaction record above and emits, per inventory item, an `observe` / `no_observation` decision plus a single shared list of sweep timestamps, partitioned into (i) *journey samples* spread across the item's bursts and (ii) *dense windows* concentrated on the most informative bursts (with a configurable rule biasing toward late, dispense-phase bursts over early retrieval). Each timestamp is tagged with the candidate items the observer should look for there, so multiple items can be searched for in a single observer call.

4b. **Sweep VLM observer.** A *single* multi-image VLM call sees the merged sweep frames and returns, for every candidate item the planner flagged on those frames, `{item_confirmed, evidence_frames, amount_remaining}`. The observer is multi-item by design (one call per round, not one call per item), so the per-frame token cost is amortised across every item the planner asked about on that frame. Amount estimation — judging how much of a jar of sauce was poured, or how many tomatoes have been removed from a bag — demands fine-grained visual reasoning that weaker models still handle poorly [cite MMVP, BLINK, V*]; we therefore pay for a strong VLM at this stage (Gemini 3.1 Pro Preview in our canonical configuration [6]) and amortise its per-call cost across all items flagged for the same sweep frames.

Stages 1–3 correspond to the lightweight "locate relevant windows" path described in Section 3; stage 4a is where a cheap language model reasons over the full session at the symbol level, and stage 4b is where the powerful VLM performs the expensive perceptual reasoning, but only on the planner-allocated frames.

Evidence frames are buried within similar detections. HOI candidates alone are not the evidence. Only the moments at which the package's remaining state is visible — the bag in hand before the pour, the carton with eggs visible, the jar on the counter — actually answer "how much remains?", and these dispensing windows form a small fraction of an item's candidate set [stat: fraction of HOI+match frames per touched item that show the package vs. a derivative]. The reason is structural: for many household items the dispensed form is visually close to the packaged product — loose vegetables, raw meat, granular staples tipped into a pan, transparent containers whose contents look the same inside and outside — so the same image-to-image similarity that ties HOI candidates to the right item also fires on every frame in which a derivative of that item is in view. Dispensing moments are not predictably anchored to the start or end of an interaction either, so uniformly subsampling within an item's candidates misses the dispensing frame as often as it lands on one [stat: hit rate of a uniform k-frame subsample against the dispensing window]. The evidence is, in this sense, buried within visually similar but uninformative detections. Stage 4 therefore uses an agentic plan–observe–replan loop that allocates a bounded frame budget over each item's candidates, inspects the returned frames, recognises when the evidence is uninformative, and requests additional views before committing an inventory update.

Plan–observe–replan (two rounds). Stage 4 is structured as a fixed two-round plan–observe–replan loop driven by the *planning agent*. The agent's action set is deliberately narrow: within a round it (i) selects sweep frames over the per-item interaction record, (ii) tags each frame with the candidate items to look for, (iii) dispatches a single multi-image observer call, and (iv) reads the observer's per-item answers; between rounds it (v) replans on items whose remaining amount, starting amount, or per-event derivative is unresolved or under-confident; and at the end of stage 4 it (vi) writes the final per-item answers back to the ledger. After round 1, the planner is re-invoked with (i) the actual timestamp ranges of frames the round-1 observer saw (not the planner's declared windows) and (ii) the per-item answers and confidences the observer returned. The round-2 planner is allowed to revisit any moment in the session — including frames already inspected — and dispatches a second sweep observer call targeting items whose remaining amount, starting amount, or per-event derivative was unresolved or under-confident in round 1. The loop terminates after round 2, or earlier if the planner declares all items resolved.

5.3 Reasoner extensions

Cross-session visual memory. Amount estimation within a single session is fundamentally under-determined when the stock container is only partially visible, or when it never appears in its final state. This extension threads a small *visual memory* across sessions: for each item the reasoner confirms, we persist a single canonical end-of-session frame (the moment the stock container is most clearly visible) together with its estimated remaining amount. On subsequent sessions the reasoner is prompted with this item's prior snapshots, in chronological order, alongside the current-session frames. The prior *numeric* estimates are withheld to avoid anchoring; only the visual evolution of the package is exposed. The intended effect is that the reasoner can compare the current fill level against the previous fill level directly, rather than re-deriving the remaining amount from scratch each session.

5.4 Baseline

A **whole-video** baseline gives the VLM the entire untrimmed video and asks it to predict remaining amounts end-to-end, with no temporal structure supplied. This is the comparator against which the *proposed system*'s cost–accuracy gain is measured.

6 Dataset

The dataset is collected from three participants cooking in their own kitchens. Each participant records first-person video using either **Meta Ray-Ban Smart Glasses** (egocentric, fisheye) or a head-mounted **Raspberry Pi Camera Module v3**, depending on hardware availability.

6.1 Collection protocol

We collect two complementary cohorts with the same recording hardware and the same item-registration and per-session annotation procedure: *in-the-wild* sessions, the primary realistic-deployment cohort, and *controlled-consumption* sessions, which broaden item-category coverage and supply paired-comparison statistical power. We first describe the registration and annotation steps that apply to both cohorts, then the protocol that differs between them.

Item registration from grocery receipts. Before any session is recorded, each item is initialized in the participant's ledger from its grocery receipt: we record the item's name, its *original package size* (in grams or count), and a product image of the item's package. These attributes are populated once when the item enters the household, sourced from the receipt (or the retailer's product page when the receipt lacks an image), and reused across every subsequent session that touches it — in both in-the-wild and controlled sessions.

Per-session annotation. For every item the participant interacted with in a session — again, in both cohorts — we record:

- **Item used** — a binary flag indicating whether any amount of the item was consumed during the session.
- **Starting amount** — the amount present at the start of the session, in grams (bulk items) or count (discrete items such as eggs).
- **Ending amount** — the amount remaining at the end of the session, in the same unit. Together with the starting amount this is the supervision signal a system must predict.
- **Consumption amount** — the session's consumed amount, equal to starting minus ending.

In-the-wild cooking sessions. Participants wear the Meta Ray-Ban Smart Glasses during their ordinary meal-prep sessions at home, with no instructions on what or how to cook. Each session is intended to capture the complete lifecycle of an inventory item within that meal — retrieval from storage, in-session use, and put-back — so that the starting and ending states of every touched item are both visible on tape. At the beginning and end of each recording, the participant places each to-be-used or just-used item on a kitchen scale and reads the weight aloud (or shows it to the camera), producing the per-item ground-truth amounts described in Section 6.1. Within the session itself, participants are explicitly *not* asked to keep the item in view or to avoid natural recording noise; hand obstruction, head turning, off-camera handling, and the usual artefacts of egocentric recording are kept in the data so that the cohort reflects the realistic deployment scenario — free-form cooking with multiple items, varied background clutter, and whatever item subset happens to be on the shelf that week. This is the primary evaluation cohort.

Controlled consumption sessions. In a controlled session the participant follows a scripted protocol that consumes a *single* preselected item at a time: the item is removed from storage, a known amount is poured/scooped/cut out, and the item is returned. Several items are cycled through in a single recording. This cohort exists to deliberately exercise items under-represented in free-form cooking (less-used condiments, opaque-packaged items, unusual shape categories) and to produce many cleanly-bounded consumption events per unit recording time, raising the event count to the level needed for paired *proposed system*-vs.-baseline comparisons at statistical significance that the wild cohort alone could not support.

6.2 Dataset summary

Table 1 reports the dataset across participants, hours of recording, weighed consumption events, and distinct tracked item instances.

Table 1: Dataset summary across participants.

	Count
Participants (recording)	5
Participants (annotated ledger)	3
Total video	29.9 h
in-the-wild	29.0 h
controlled	0.9 h
Consumption events (weighed)	460
in-the-wild	388
controlled	72
Distinct tracked item instances	150

Across the annotated ledgers, the 150 tracked item instances fall into four shape categories (Table 2 breaks this down with examples): SOLID 39, DISCRETE 36, GRANULAR 33, FLUID 20, with 22 instances not yet categorised.

6.3 Item variety

Each item in the dataset is tagged with a *shape category* — one of {FLUID, GRANULAR, SOLID, DISCRETE} — along with a *transparency* flag indicating whether the package is see-through. These attributes are informative because they govern how an amount is estimated: fluid and granular items in transparent packaging permit fill-level reading, discrete items (eggs, yogurt cups) permit counting, and opaque containers must be inferred from pour/scoop duration or weighing context. Table 2 summarises the current item inventory.

6.4 Longitudinal coverage

The same item often persists across many sessions before it is exhausted, which lets us evaluate *how a system tracks one package*

Table 2: Item variety across all annotated ledgers, grouped by shape category. Counts are item *instances* (one row per purchase).

Category	Items	Examples
Fluid	20	olive oil, milk, soy sauce
Granular	33	rice, pasta, cereal
Solid	39	garlic, steak, tofu
Discrete	36	eggs, yogurt cups, bagels
Unlabelled	22	
Total	150	

over its full lifecycle rather than only how it does on isolated sessions. Across the annotated ledgers, item lifecycles span the distribution shown in Table 3: 36 item-instances appear in at least 5 distinct sessions, 12 in at least 10, and the longest-running items (eggs, sliced cheese, cereal, oats, milk gallons) are observed across 13–18 sessions each. This long-tail subset is the substrate for the per-item lifecycle evaluation described in Section 7.5.

Table 3: Item lifecycle distribution across annotated ledgers. "Sessions" is the number of distinct sessions in which a given item instance was logged as consumed.

Sessions per item	Item instances	Cumulative (≥)
≥1	107	107
≥3	61	61
≥5	36	36
≥10	12	12
≥15	3	3

7 Benchmark and Evaluation Plan

7.1 Benchmark structure

Videos are organized into *sessions* (meal-prep episodes).

Task. Given (i) an untrimmed egocentric session video and (ii) the participant's current inventory ledger snapshot (Section 5.1), the system outputs, for every item the participant interacted with, an estimate of the item's *remaining amount* at session end — equivalently the *consumption delta* relative to the ledger's starting amount. The ground-truth label, for every touched item, is the (*starting amount*, *ending amount*) pair from Section 6, against which either form of the prediction is scored.

Evaluation axes. A run is characterised by two scalars per session: amount-estimation *accuracy* (Section 7.3) and VLM *cost* (Section 7.4). The headline result (Section 8.2) plots accuracy against cost as a Pareto frontier; a system that wins on one axis while losing on the other is not preferred unless the trade is explicitly characterised along that frontier.

7.2 Evaluation cohorts

We evaluate the two collection flavours for different purposes.

In-the-wild cooking videos. These are our primary cohort and the one that reflects the deployed use case. On this cohort we report *both* (i) amount-estimation accuracy (CNPE, Section 7.3) and (ii) VLM token consumption per session (Section 7.4). Target scale at submission: ∼20 h of video and ∼400 consumption events across 3 participants (numbers to be finalised).

Controlled consumption videos. We use the controlled cohort to evaluate VLM amount-estimation accuracy over a broader, deliberately diversified set of food items (see Section 6). The item mix here spans shape categories and packaging transparencies that are under-represented in any one participant's natural cooking, which lets us measure how *proposed system*'s accuracy gain over baselines generalises across item types. Target scale: ∼2 h of video and ∼300 consumption events; we report CNPE on this cohort with paired per-event comparisons so that *proposed system*-vs.-baseline differences can be tested at statistical significance.

7.3 Primary metric: CNPE (accuracy)

We measure accuracy with **Capacity-Normalized Percentage Error (CNPE)**:

$$\text{CNPE} = \frac{|\text{prediction} - \text{GT}|}{W_0} \times 100,$$

where W_0 is the item's original package size (capacity). Normalizing by W_0 makes errors comparable across items of very different sizes: a 50 g error on a 500 g jar (10% CNPE) is treated the same as a 50 g error on a 50 g packet (100% CNPE). CNPE can be computed equivalently on the remaining amount or the used amount.

7.4 Primary metric: VLM token consumption (cost)

We report total cloud-VLM token cost per session as a USD figure under the locked pricing convention (Section 7.7), priced across three token classes:

- **Input tokens**: prompt text, the in-prompt ledger snapshot, and the per-frame image tokens fed to the observer.
- **Output tokens**: the observer's per-item answers and the planner's plan-of-frames.
- **Thinking tokens**: model-internal chain-of-thought tokens, which both Gemini and GPT-5 bill at the output rate.

Cloud planner-LLM tokens are included in the *proposed system* cost; the whole-video baseline calls only the VLM and incurs no planner cost.

Edge compute is excluded from the USD axis, characterised separately. The Hands23 HOI detector, OWLv2 scene tagger, and DINOv3 identification ViT all run on the household endpoint (Section 5), once per session, and are not billed per call. The headline cost claim of *proposed system* is about *recurring API dollars* — the per-session out-of-pocket cost a household incurs as it records more video — so edge compute is excluded from the USD axis. We exclude rather than convert because edge hardware varies across deployments (a phone, a home workstation, a local server), and edge cost amortises over session lifetime in a way the cloud per-token model does not; pricing GPU-hours into a single USD axis

would conflate two fundamentally different cost regimes the system is designed to trade between. The edge pipeline is instead characterised on its own terms: per-stage *peak GPU memory* and *inference latency* (seconds per session-hour of video) measured on the deployment hardware, reported alongside the headline cost table so the household-side resource footprint is auditable.

7.5 Cross-cut analyses

The two headline metrics (CNPE, token cost) are reported as session-level aggregates, but the questions that drive system design are sharper than a single mean. We additionally report cross-cut breakdowns of the same underlying per-event predictions along four axes — shape category, packaging transparency, item lifecycle, and lifecycle stage — with each cross-cut computed separately within the in-the-wild and controlled cohorts so that cohort effects are never silently folded into category effects.

Per-class CNPE.. CNPE is broken down by item shape category (FLUID, GRANULAR, SOLID, DISCRETE) and crossed with packaging transparency. The hypothesis is that the *proposed system*'s gain over the baseline concentrates on classes where amount reasoning is locally hard but *visible* given the right frame — fill levels in transparent fluids, residue in granulars, missing items in discrete counts — whereas opaque-fluid and irregular-solid estimation remain hard regardless of how the observation windows are chosen. Reporting per-class CNPE makes the source of any aggregate gain auditable and exposes packaging types where future work is most needed. The per-class event counts in Table 2 (20–39 instances per category, with multiple consumption events per instance) are sufficient to support per-class means with bootstrap intervals.

Item-lifecycle CNPE.. Inventory tracking is intrinsically a longitudinal task: the same package is observed across many sessions until exhausted. For the long-tail item subset characterised in Section 6.4 (32 item-instances with $\geq$5 sessions, 11 with $\geq$10), we report a per-item CNPE *trajectory* indexed by session number, alongside a single per-item lifecycle CNPE (mean across that item's sessions). Two trajectory shapes matter and we will distinguish them: (i) session-by-session error that stays roughly flat, indicating the system re-anchors against the current frame and earlier session errors do not compound; and (ii) error that grows monotonically as the package empties, indicating that low-fill states are a systematic failure mode and motivating the cross-session visual memory extension (Section 5.2). For items observed all the way to a "finished" event, we additionally report end-of-life cumulative drift (predicted total used minus actual, normalised by capacity).

Lifecycle-stage CNPE.. Independently of which item-instance they came from, consumption events are bucketed by the *lifecycle stage* of the item at the start of the session — EARLY-LIFE ($\geq$70% of capacity remaining), MID-LIFE (between 20% and 70%), and LATE-LIFE ($\leq$20%) — and CNPE is reported per bucket. The hypothesis is that late-life sessions, where the package is mostly empty and small absolute changes correspond to large CNPE, dominate aggregate error in a way that motivates the cross-session visual memory extension (Section 5.2). Splitting events by stage rather than only by item-instance makes this driver visible across the whole dataset, including items not yet long enough for a full trajectory.

7.6 Experiment matrix

Every cell in the matrix below is reported under both cross-cut breakdowns from Section 7.5 (per-class and item-lifecycle), so that the headline CNPE/token comparisons can be audited along the dimensions most likely to drive system design.

- **Controlled vs. natural sessions.** We expect amount-estimation error to be lower on controlled sessions (single item, scripted consumption) and will report the two cohorts separately.
- **Baseline vs. *proposed system*.** Whole-video baseline and the *proposed system* (planner + sweep observer with iterative replan per Section 5), using the same VLM backends for a fair comparison.
- **Ablations within the *proposed system*.** (i) Number of plan–observe rounds (1 vs. 2 vs. unbounded under a session-level frame budget); (ii) frame-budget allocation policy (journey-only, dense-only, mixed; late-burst-biased vs. uniform); (iii) planner evidence rendering (per-item temporal segments vs. chronological timeline vs. activity blocks); (iv) backbone VLM for the observer (GPT-5, Gemini 3 Pro / 2.5 Pro / Flash, Gemma) under matched frame-budget caps so the comparison isolates VLM choice from frame count.
- **Extension (Section 5.2).** We will additionally report *cross-session visual memory*: the *proposed system* with prior end-of-session snapshots threaded into the reasoner prompt, versus the same *proposed system* without — evaluated specifically on the long-tail items of Table 3 where the trajectory metric (Section 7.5) makes any cross-session benefit visible. The extension is evaluated on the same CNPE/token axes so its cost/accuracy trade-off against the base *proposed system* is directly comparable.
- **Scalability.** As more participants and sessions come online, we will re-run the full matrix and report metric stability as the experiment set grows.

7.7 Locked evaluation definitions

To make the small-scale pilots and the final-cohort runs directly comparable, three definitions are pinned for the remainder of the project. Any later change to these is treated as a methodological revision and the affected runs are re-executed.

Frozen cohort lists. The wild and controlled cohorts are specified as explicit session-ID lists per participant (Appendix **??**, to be added). New sessions recorded after the freeze date are excluded from the headline numbers and may appear only as out-of-distribution diagnostics. The current pilot cohort is the 10-session canonical `kailai` natural-cooking subset; the final cohort substitutes the larger frozen list under the same protocol.

Frozen pricing convention. Billable cost per session is reported in USD using Gemini 3 Pro paid-tier pricing ($1.25/M input, $10/M output, thinking tokens charged at output rate), with images converted at 1 frame = 260 input tokens. The same pricing convention is applied to the whole-video baseline and the *proposed system* so that cost comparisons are not confounded by per-comparator pricing choices. Planner LLM cost is included only in the *proposed system* and extension rows.

Frozen metric definitions. Three metrics constitute the headline:

- **CNPE per event** (Section 7.3), reported as the mean over GT consumption events.
- **Recall** = the fraction of GT consumption events for which the system produced a non-null amount estimate. Items the planner explicitly skipped, items the observer marked as not-confirmed, and items never proposed in the first place all count as missed.
- **Fused CNPE+recall** = the per-event mean of CNPE with missed events charged at CNPE = 100. This single scalar is the y-axis of the headline cost–accuracy figure. The failure floor of 100 corresponds to a full-package error and prevents systems from improving the headline by skipping uncertain items.

8 Results

This section is a skeleton. Each figure and table below is defined here so that experiments executed between now and the freeze date populate specific cells rather than producing orphaned runs. Figures and tables will be filled as experiments complete.

8.1 Setup

Final cohort sizes (sessions, hours, consumption events) per participant, broken out by wild vs. controlled. One row per comparator (whole-video baseline, *proposed system* R1, *proposed system* R1+R2, *proposed system*+memory) listing the locked run configuration: planner LLM, observer VLM, frame-budget cap, evidence rendering mode, burst-coverage rule, thinking depth. Pricing convention restated from Section 7.7.

8.2 Headline: cost–accuracy frontier

Figure (central): Pareto plot. x-axis = USD/session (log scale); y-axis = fused CNPE+recall (lower is better). One marker per (comparator × frame-budget) cell. Curves: whole-video baseline (single point), *proposed system* frontier (sweep over frame budget and round count). Iso-cost guides at \$0.10, \$0.50, \$1.00. Extension points appended in red when implemented.

Table 4 (placeholder): aggregate CNPE, recall, and \$/session for each comparator on the wild and controlled cohorts.

8.3 Cross-cut analyses

Two panels populate the breakdowns specified in Section 7.5:

- **Per-class CNPE** bars: shape category (FLUID/GRANULAR/SOLID/DISCRETE) × comparator, crossed with packaging transparency. Tests the "gain concentrates on visible-but-locally-hard classes" hypothesis from Section 7.5.
- **Lifecycle trajectory grid:** small multiples, one panel per long-tail item from Table 3, x = session index, y = CNPE, two lines per panel (whole-video baseline / *proposed system*). End-of-life cumulative drift reported in an adjacent table for items observed to "finished".

8.4 Ablations

Each ablation produces a small Pareto subplot sharing axes with the headline figure. (i) Round count (R1 vs. R1+R2 vs. unbounded under a session-level frame budget). (ii) Frame budget allocation policy (journey-only / dense-only / mixed; late-burst vs. uniform). (iii) Planner evidence rendering (`blocks` vs. `chrono` vs. `segments`). (iv) Observer-backbone swap (GPT-5, Gemini 3 Pro, Gemini 2.5 Pro/Flash, Gemma) at matched frame-budget caps.

8.5 Failure analysis

2–3 hand-picked sessions with figure-strip illustrations: session timeline, frames the observer actually saw, predicted vs. true amount, one-sentence diagnosis. Each case is mapped to either the cross-session memory extension or flagged as an open problem.

8.6 Extension

Cross-session visual memory results: matched-frame-budget comparison against base *proposed system*, evaluated on the Table 3 long-tail items so the lifecycle trajectories from Section 8.3 can show the benefit.

Table 4: (Placeholder.) Headline CNPE, recall, and \$/session per comparator on the frozen cohorts.

	Wild			Controlled		
Comparator	CNPE	Recall	\$/sess	CNPE	Recall	\$/sess
Whole-video baseline	–	–	–	–	–	–
proposed system (R1)	–	–	–	–	–	–
proposed system (R1+R2)	–	–	–	–	–	–
proposed system+mem	–	–	–	–	–	–

9 Related Work

long video understanding ego: anchored by HOI and scene [5], [9]
agentic: avp, ava,
object state change
Datasets/benchmark on Object States: follow MASCATO [20]

References

[1] Dima Damen, Hazel Doughty, Giovanni Maria Farinella, Antonino Furnari, Evangelos Kazakos, Jian Ma, Davide Moltisanti, Jonathan Munro, Toby Perrett, Will Price, and Michael Wray. 2022. Rescaling Egocentric Vision: Collection, Pipeline and Challenges for EPIC-KITCHENS-100. 130, 1 (2022), 33–55. doi:10.1007/s11263-021-01531-2

[2] Ahmad Darkhalil, Dandan Shan, Bin Zhu, Jian Ma, Amlan Kar, Richard Higgins, Sanja Fidler, David Fouhey, and Dima Damen. 2022. *EPIC-KITCHENS VISOR Benchmark: VIdeo Segmentations and Object Relations.* arXiv:2209.13064 [cs] doi:10.48550/arXiv.2209.13064

[3] Yue Fan, Xiaojian Ma, Rongpeng Su, Jun Guo, Rujie Wu, Xi Chen, and Qing Li. 2025. *Embodied VideoAgent: Persistent Memory from Egocentric Videos and Embodied Sensors Enables Dynamic Scene Understanding.* arXiv:2501.00358 [cs] doi:10.48550/arXiv.2501.00358

[4] Yue Fan, Xiaojian Ma, Rujie Wu, Yuntao Du, Jiaqi Li, Zhi Gao, and Qing Li. 2025. VideoAgent: A Memory-Augmented Multimodal Agent for Video Understanding. In *Computer Vision – ECCV 2024*, Aleš Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol (Eds.). Vol. 15080. Springer Nature Switzerland, 75–92. doi:10.1007/978-3-031-72670-5_5

[5] Gabriele Goletto, Tushar Nagarajan, Giuseppe Averta, and Dima Damen. 2024. *AMEGO: Active Memory from Long EGOcentric Videos.* arXiv:2409.10917 [cs] doi:10.48550/arXiv.2409.10917

[6] Google. 2026. *Gemini API Pricing*. Google. https://ai.google.dev/gemini-api/docs/pricing Per-token input/output pricing for Gemini 3 Pro and related Gemini API models..

[7] Google. 2025. Gemini 3 Pro Model Card. https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Pro-Model-Card.pdf. Model card update: December 2025.

[8] Kristen Grauman, Andrew Westbury, Eugene Byrne, Zachary Chavis, Antonino Furnari, Rohit Girdhar, Jackson Hamburger, Hao Jiang, Miao Liu, Xingyu Liu, et al. 2022. Ego4d: Around the world in 3,000 hours of egocentric video. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 18995–19012.

[9] Zeyi Huang, Yuyang Ji, Xiaofang Wang, Nikhil Mehta, Tong Xiao, Donghyun Lee, Sigmund Vanvalkenburgh, Shengxin Zha, Bolin Lai, Licheng Yu, et al. 2025. Building a mind palace: Structuring environment-grounded semantic graphs for effective long video analysis with llms. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 24169–24179.

[10] Junlong Li, Huaiyuan Xu, Sijie Cheng, Kejun Wu, Kim-Hui Yap, Lap-Pui Chau, and Yi Wang. 2025. *Building Egocentric Procedural AI Assistant: Methods, Benchmarks, and Challenges.* arXiv:2511.13261 [cs] doi:10.48550/arXiv.2511.13261

[11] Tushar Nagarajan, Yanghao Li, Christoph Feichtenhofer, and Kristen Grauman. 2020. Ego-topo: Environment affordances from egocentric video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 163–172.

[12] OpenAI. 2026. *OpenAI API Pricing*. OpenAI. https://openai.com/api/pricing/ Per-token input/output pricing for GPT-5 family models..

[13] Rohith Peddi, Shivvrat Arya, Bharath Challa, Likhitha Pallapothula, Akshay Vyas, Bhavya Gouripeddi, Jikai Wang, Qifan Zhang, Vasundhara Komaragiri, Eric Ragan, Nicholas Ruozzi, Yu Xiang, and Vibhav Gogate. 2024. *CaptainCook4D: A Dataset for Understanding Errors in Procedural Activities.* arXiv:2312.14556 [cs] doi:10.48550/arXiv.2312.14556

[14] Toby Perrett, Ahmad Darkhalil, Saptarshi Sinha, Omar Emara, Sam Pollard, Kranti Parida, Kaiting Liu, Prajwal Gatti, Siddhant Bansal, Kevin Flanagan, Jacob Chalk, Zhifan Zhu, Rhodri Guerrier, Fahd Abdelazim, Bin Zhu, Davide Moltisanti, Michael Wray, Hazel Doughty, and Dima Damen. 2025. *HD-EPIC: A Highly-Detailed Egocentric Video Dataset.* arXiv:2502.04144 [cs] doi:10.48550/arXiv.2502.04144

[15] Aniket Rege, Arka Sadhu, Yuliang Li, Kejie Li, Ramya Korlakai Vinayak, Yuning Chai, Yong Jae Lee, and Hyo Jin Kim. 2026. Agentic Very Long Video Understanding. *arXiv preprint arXiv:2601.18157* (2026).

[16] Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. 2025. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267* (2025).

[17] Yihong Sun, Xinyu Yang, Jennifer J. Sun, and Bharath Hariharan. 2025. *Tracking and Understanding Object Transformations.* arXiv:2511.04678 [cs] doi:10.48550/arXiv.2511.04678

[18] Masatoshi Tateno, Takuma Yagi, Ryosuke Furuta, and Yoichi Sato. 2025. Learning multiple object states from actions via large language models. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 9555–9565.

[19] Mahiro Ukai, Shuhei Kurita, and Nakamasa Inoue. 2025. STATUS Bench: A Rigorous Benchmark for Evaluating Object State Understanding in Vision-Language Models. In *Proceedings of the 33rd ACM International Conference on Multimedia*. 4718–4727.

[20] Ziyang Wang, Honglu Zhou, Shijie Wang, Junnan Li, Caiming Xiong, Silvio Savarese, Mohit Bansal, Michael S Ryoo, and Juan Carlos Niebles. 2025. Active Video Perception: Iterative Evidence Seeking for Agentic Long Video Understanding. *arXiv preprint arXiv:2512.05774* (2025).

[21] Yuxuan Yan, Shiqi Jiang, Ting Cao, Yifan Yang, Qianqian Yang, Yuanchao Shu, Yuqing Yang, and Lili Qiu. 2025. AVA: Towards Agentic Video Analytics with Vision Language Models. *arXiv preprint arXiv:2505.00254* (2025).

[22] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[23] Parnian Zameni, Yuhan Shen, and Ehsan Elhamifar. 2025. Moscato: Predicting multiple object state change through actions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 11600–11611.

[24] Xiaoyi Zhang, Zhaoyang Jia, Zongyu Guo, Jiahao Li, Bin Li, Houqiang Li, and Yan Lu. 2025. Deep video discovery: Agentic search with tool use for long-form video understanding. *arXiv preprint arXiv:2505.18079* (2025).